Deep Learning Laboratory

Lab Practice IV (414447) – Practical Journal Data

BE (Information Technology), 2019 Course, Savitribai Phule Pune University

Name _____

Roll No _____

PRN _____

Class _____

Contents

No.	Practical	Notebook file
1	Study of Deep Learning Packages: TensorFlow, Keras, Theano and PyTorch	DL_Ass__01.ipynb
2	Implementing Feedforward Neural Networks with Keras and TensorFlow (MNIST)	DL_LAB_ASS_02.ipynb
3	Image Classification Model (CNN, MNIST) in 4 Stages	DL_LAB_ASS_03.ipynb
4	Autoencoder for Anomaly Detection (Credit Card Fraud)	DL_ASS4.ipynb
5	Continuous Bag of Words (CBOW) Model	DL_ASS5.ipynb
6	Transfer Learning of CNN Architectures (VGG16, tf_flowers)	Assignemnt_6.ipynb

Each practical below reproduces its notebook in order: markdown notes, code cells, and the outputs and plots as saved in the notebook. Very long package-installation logs in Practical 1 are shortened (marked in the text). All other outputs are complete.

Practical 1: Study of Deep Learning Packages: TensorFlow, Keras, Theano and PyTorch

Notebook: DL_Ass__01.ipynb **Syllabus stages:** Installation and test program for each package; Document the distinct features and functionality of the packages

Tensorflow Test Program

Installation of Tensorflow

In [2]
```
pip install tensorflow
```

Out
```
Collecting tensorflow
  Downloading tensorflow-2.10.0-cp39-cp39-win_amd64.whl (455.9 MB)
Collecting absl-py>=1.0.0
  Downloading absl_py-1.3.0-py3-none-any.whl (124 kB)
Collecting tensorflow-estimator<2.11,>=2.10.0
  Downloading tensorflow_estimator-2.10.0-py2.py3-none-any.whl (438 kB)
Requirement already satisfied: wrapt>=1.11.0 in c:\users\poona\anaconda3\lib\site-packages (from tensorflow) (1.12.1)
Collecting libclang>=13.0.0
... [56 more lines of the installation log omitted]
```

In [3]
```
import tensorflow as tf
```

Print out Tensorflow version

In [5]
```
print(tf.__version__)
```

Out
```
2.10.0
```

In [6]
```
print(tf.reduce_sum(tf.random.normal([1000,1000])))
```

Out
```
tf.Tensor(-525.7693, shape=(), dtype=float32)
```

In [4]
```
pip install keras
```

Out
```
Requirement already satisfied: keras in c:\users\poona\anaconda3\lib\site-packages (2.10.0)
Note: you may need to restart the kernel to use updated packages.
```

Keras Test Program

In [7]
```
from tensorflow import keras
```

In [8]
```
from keras import datasets
```

Loading MNIST Data

In [9]
```
(train_images,train_labels),(test_images,test_labels)=datasets.mnist.load_data()
```

Out
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 [==========] - 1s 0us/step
```

Check the dataset loaded

In [10]
```
train_images.shape,test_images.shape
```

Out
```
((60000, 28, 28), (10000, 28, 28))
```

Theano Test Program

In [13]
```
pip install theano
```

Out
```
Collecting theano
  Downloading Theano-1.0.5.tar.gz (2.8 MB)
Requirement already satisfied: numpy>=1.9.1 in c:\users\poona\anaconda3\lib\site-packages (from theano) (1.21.5)
Requirement already satisfied: scipy>=0.14 in c:\users\poona\anaconda3\lib\site-packages (from theano) (1.7.3)
Requirement already satisfied: six>=1.9.0 in c:\users\poona\anaconda3\lib\site-packages (from theano) (1.16.0)
Building wheels for collected packages: theano
  Building wheel for theano (setup.py): started
```

```
  Building wheel for theano (setup.py): finished with status 'done'
... [6 more lines of the installation log omitted]
```

In [14]
```
import numpy
```

In [15]
```
import theano.tensor as T
```

Out
```
WARNING (theano.tensor.blas): Using NumPy C-API based implementation for BLAS functions.
```

In [16]
```
from theano import function
```

Addition of two scalars

Declaring Two Variables

In [17]
```
x = T.dscalar('x')
```

In [18]
```
y = T.dscalar('y')
```

summing up the two numbers

In [24]
```
z = x + y
```

In [20]
```
f = function([x,y],z)
```

In [21]
```
f(5,7)
```

Out
```
array(12.)
```

PyTorch Test Program

In [23]
```
pip install torch
```

Out
```
Collecting torch
  Downloading torch-1.12.1-cp39-cp39-win_amd64.whl (161.8 MB)
Requirement already satisfied: typing-extensions in c:\users\poona\anaconda3\lib\site-packages (from torch) (4.1.1)
Installing collected packages: torch
Successfully installed torch-1.12.1
Note: you may need to restart the kernel to use updated packages.
```

Importing torch

In [25]
```
import torch
```

In [26]
```
import torch.nn as nn
```

Print out PyTorch version

In [27]
```
print(torch.__version__)
```

Out
```
1.12.1+cpu
```

Practical 2: Implementing Feedforward Neural Networks with Keras and TensorFlow (MNIST)

Notebook: DL_LAB_ASS_02.ipynb **Syllabus stages:** Import the necessary packages; Load the training and testing data; Define the network architecture using Keras; Train the model using SGD; Evaluate the network; Plot the training loss and accuracy

Importing Libraries

In [3]

```
import tensorflow as tf
from tensorflow import keras
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import random
%matplotlib inline
```

Loading and preparing the data

In [4]

```
#import dataset and split into train and test
mnist = tf.keras.datasets.mnist
(x_train,y_train),(x_test,y_test) = mnist.load_data()
```

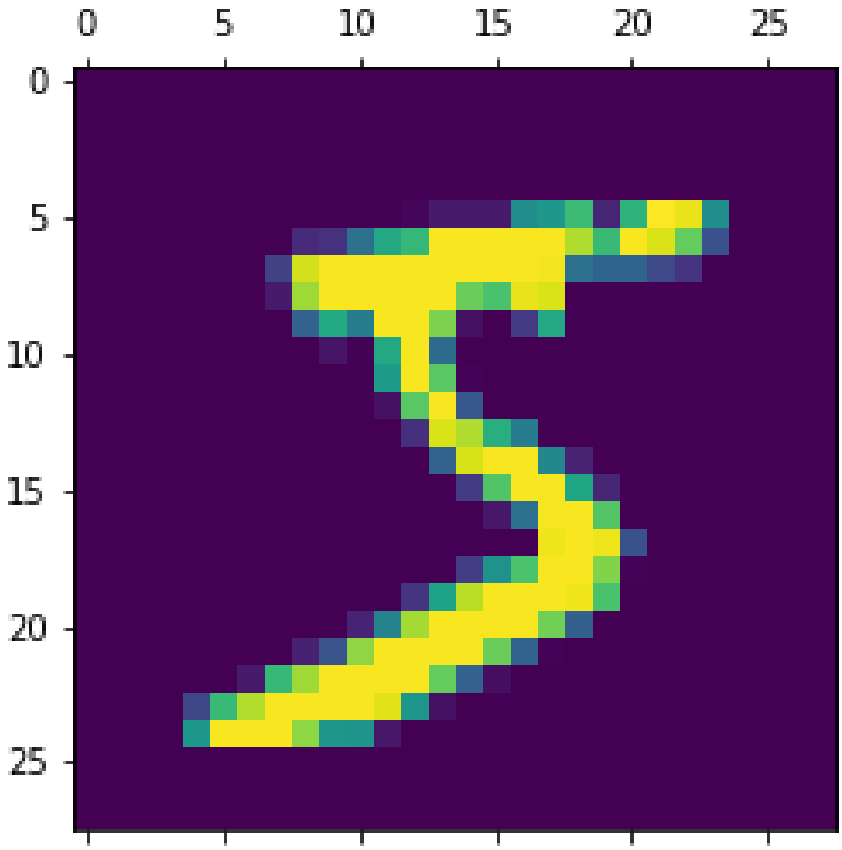
In [11]

```
#to see how first image looks
plt.matshow(x_train[0])
```

Out

```
<matplotlib.image.AxesImage at 0x1c823884a00>
```

Out (plot)



In [12]

```
#normalising the images by scaling the pixel intensities to the range 0 to 1
x_train = x_train / 255
x_test = x_test / 255
```

Creating the Model

In [13]

```
model = keras.Sequential([
```

```
    keras.layers.Flatten(input_shape=(28,28)),
    keras.layers.Dense(128,activation = 'relu'),
    keras.layers.Dense(10,activation = 'softmax')
])
```

In [14]

```
model.summary()
```

Out

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=========
 flatten (Flatten)           (None, 784)               0

 dense (Dense)               (None, 128)               100480

 dense_1 (Dense)             (None, 10)                1290

==========
Total params: 101,770
Trainable params: 101,770
Non-trainable params: 0
_________________________________________________________________
```

Compile the model

In [16]

```
model.compile(optimizer = 'sgd',loss = 'sparse_categorical_crossentropy', metrics = ['accuracy'])
```

Train the Model

In [17]

```
history = model.fit(x_train,y_train,validation_data = (x_test,y_test), epochs = 10)
```

Out

```
Epoch 1/10
1875/1875 [==========] - 9s 4ms/step - loss: 0.6522 - accuracy: 0.8363 - val_loss: 0.3547 - val_accuracy: 0.9041
Epoch 2/10
1875/1875 [==========] - 7s 4ms/step - loss: 0.3354 - accuracy: 0.9068 - val_loss: 0.2931 - val_accuracy: 0.9151
Epoch 3/10
1875/1875 [==========] - 8s 4ms/step - loss: 0.2876 - accuracy: 0.9189 - val_loss: 0.2598 - val_accuracy: 0.9264
Epoch 4/10
1875/1875 [==========] - 8s 4ms/step - loss: 0.2590 - accuracy: 0.9276 - val_loss: 0.2383 - val_accuracy: 0.9336
Epoch 5/10
1875/1875 [==========] - 8s 4ms/step - loss: 0.2372 - accuracy: 0.9336 - val_loss: 0.2248 - val_accuracy: 0.9365
Epoch 6/10
1875/1875 [==========] - 8s 4ms/step - loss: 0.2196 - accuracy: 0.9380 - val_loss: 0.2070 - val_accuracy: 0.9405
Epoch 7/10
1875/1875 [==========] - 8s 4ms/step - loss: 0.2046 - accuracy: 0.9432 - val_loss: 0.1943 - val_accuracy: 0.9449
Epoch 8/10
1875/1875 [==========] - 8s 4ms/step - loss: 0.1913 - accuracy: 0.9460 - val_loss: 0.1850 - val_accuracy: 0.9474
Epoch 9/10
1875/1875 [==========] - 8s 4ms/step - loss: 0.1797 - accuracy: 0.9494 - val_loss: 0.1730 - val_accuracy: 0.9499
Epoch 10/10
1875/1875 [==========] - 8s 4ms/step - loss: 0.1692 - accuracy: 0.9521 - val_loss: 0.1642 - val_accuracy: 0.9522
```

Evaluate the Model

In [19]

```
test_loss,test_acc = model.evaluate(x_test,y_test)
print("Loss = %.3f" %test_loss)
print("Accuracy = %.3f" %test_acc)
```

Out

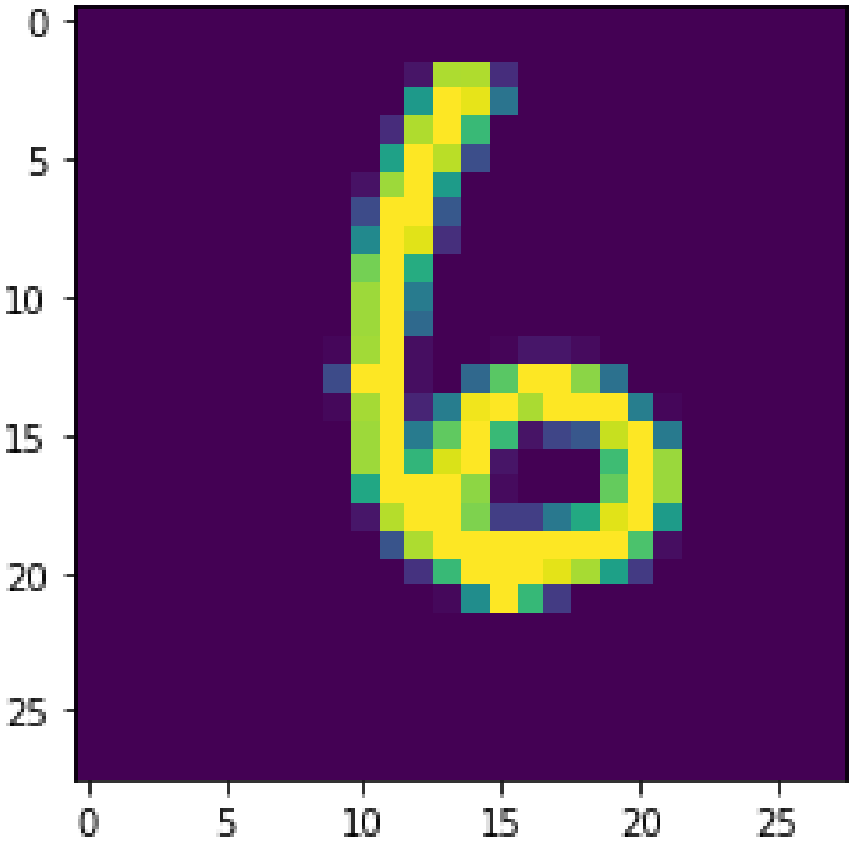
```
313/313 [==========] - 1s 3ms/step - loss: 0.1642 - accuracy: 0.9522
Loss = 0.164
Accuracy = 0.952
```

Making Prediction on New Data

In [23]

```
n = random.randint(0,9999)
plt.imshow(x_test[n])
plt.show()
```

Out (plot)



Confusion Matrix

In [25]

```
test_predict = model.predict(x_test)
#get classification labels
test_predict_labels = np.argmax(test_predict,axis = 1)
confusion_matrix = tf.math.confusion_matrix(labels = y_test, predictions = test_predict_labels)
print('Confusion Matrix of the Test Set :\n' , confusion_matrix)
```

Out

```
313/313 [==========] - 1s 2ms/step
Confusion Matrix of the Test Set :
 tf.Tensor(
[[ 960    0    1    2    0    5    7    2    2    1]
 [   0 1115    3    2    1    1    3    2    8    0]
 [   8    1  981    8    7    1    8    7    9    2]
 [   0    0   10  960    1   15    0   10    8    6]
 [   1    0    4    0  936    0    9    3    5   24]
 [   9    1    2   19    4  833   11    0    8    5]
 [   9    3    5    1   10   11  916    1    2    0]
 [   1    6   18    6    5    2    0  966    3   21]
 [   5    1    6   16    7   10   13    8  900    8]
 [   6    6    1   11   17    3    1    7    2  955]], shape=(10, 10), dtype=int32)
```

Practical 3: Image Classification Model (CNN, MNIST) in 4 Stages

Notebook: DL_LAB_ASS_03.ipynb **Syllabus stages:** Loading and preprocessing the image data; Defining the model's architecture; Training the model; Estimating the model's performance

Importing libraries

In [3]

```
import numpy as np
import pandas as pd
import random
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten,Conv2D,Dense,MaxPooling2D
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.datasets import mnist
```

Loading and preparing the data

In [8]

```
(X_train,y_train),(X_test,y_test) = mnist.load_data()
```

In [9]

```
print(X_train.shape)
```

Out

```
(60000, 28, 28)
```

In [10]

```
X_train[0].min(), X_train[0].max()
```

Out

```
(0, 255)
```

In [11]

```
X_train = (X_train - 0.0) / (255.0 - 0.0)
X_test = (X_test - 0.0) / (255.0 - 0.0)
X_train[0].min(), X_train[0].max()
```

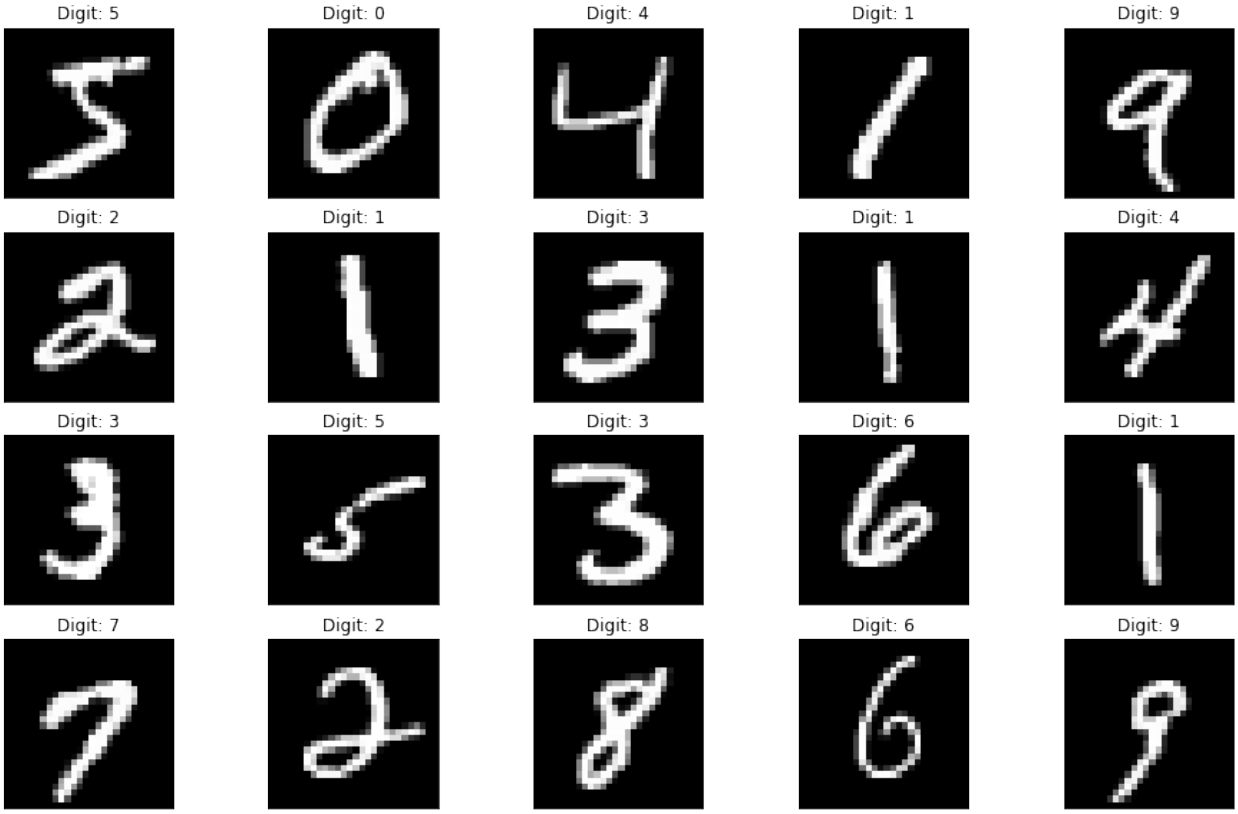
Out

```
(0.0, 1.0)
```

In [12]

```
def plot_digit(image, digit, plt, i):
  plt.subplot(4, 5, i + 1)
  plt.imshow(image, cmap=plt.get_cmap('gray'))
  plt.title(f"Digit: {digit}")
  plt.xticks([])
  plt.yticks([])
plt.figure(figsize=(16, 10))
for i in range(20):
  plot_digit(X_train[i], y_train[i], plt, i)
plt.show()
```

Out (plot)



In [13]

```
X_train = X_train.reshape((X_train.shape + (1,)))
X_test = X_test.reshape((X_test.shape + (1,)))
y_train[0:20]
```

Out

```
array([5, 0, 4, 1, 9, 2, 1, 3, 1, 4, 3, 5, 3, 6, 1, 7, 2, 8, 6, 9],
      dtype=uint8)
```

Creating the Model

In [14]

```
model = Sequential([
 Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
 MaxPooling2D((2, 2)),
 Flatten(),
 Dense(100, activation="relu"),
 Dense(10, activation="softmax")
])
```

In [15]

```
optimizer = SGD(learning_rate=0.01, momentum=0.9)
model.compile(
 optimizer=optimizer,
 loss="sparse_categorical_crossentropy",
 metrics=["accuracy"]
)
model.summary()
```

Out

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
==========
 conv2d (Conv2D)             (None, 26, 26, 32)        320

 max_pooling2d (MaxPooling2D  (None, 13, 13, 32)       0
 )

 flatten (Flatten)           (None, 5408)              0

 dense (Dense)               (None, 100)               540900

 dense_1 (Dense)             (None, 10)                1010

==========
Total params: 542,230
Trainable params: 542,230
Non-trainable params: 0
_________________________________________________________________
```

Train the Model

In [35]

```
history = model.fit(X_train, y_train,validation_data = (x_test,y_test), epochs=10)
```

Out

```
Epoch 1/10
1875/1875 [==========] - 40s 21ms/step - loss: 2.6328e-04 - accuracy: 1.0000 - val_loss: 23.2731 - val_accuracy: 0.9804
Epoch 2/10
1875/1875 [==========] - 45s 24ms/step - loss: 2.4964e-04 - accuracy: 1.0000 - val_loss: 23.2049 - val_accuracy: 0.9805
Epoch 3/10
1875/1875 [==========] - 44s 23ms/step - loss: 2.1729e-04 - accuracy: 1.0000 - val_loss: 23.0110 - val_accuracy: 0.9801
Epoch 4/10
1875/1875 [==========] - 47s 25ms/step - loss: 2.0789e-04 - accuracy: 1.0000 - val_loss: 22.9585 - val_accuracy: 0.9807
Epoch 5/10
1875/1875 [==========] - 50s 27ms/step - loss: 1.9068e-04 - accuracy: 1.0000 - val_loss: 23.5215 - val_accuracy: 0.9815
Epoch 6/10
1875/1875 [==========] - 46s 25ms/step - loss: 1.7933e-04 - accuracy: 1.0000 - val_loss: 23.8986 - val_accuracy: 0.9806
Epoch 7/10
1875/1875 [==========] - 45s 24ms/step - loss: 1.6882e-04 - accuracy: 1.0000 - val_loss: 24.3135 - val_accuracy: 0.9804
Epoch 8/10
1875/1875 [==========] - 44s 24ms/step - loss: 1.6027e-04 - accuracy: 1.0000 - val_loss: 23.9603 - val_accuracy: 0.9805
Epoch 9/10
1875/1875 [==========] - 43s 23ms/step - loss: 1.5045e-04 - accuracy: 1.0000 - val_loss: 23.7505 - val_accuracy: 0.9813
Epoch 10/10
1875/1875 [==========] - 43s 23ms/step - loss: 1.4534e-04 - accuracy: 1.0000 - val_loss: 23.6390 - val_accuracy: 0.9812
```

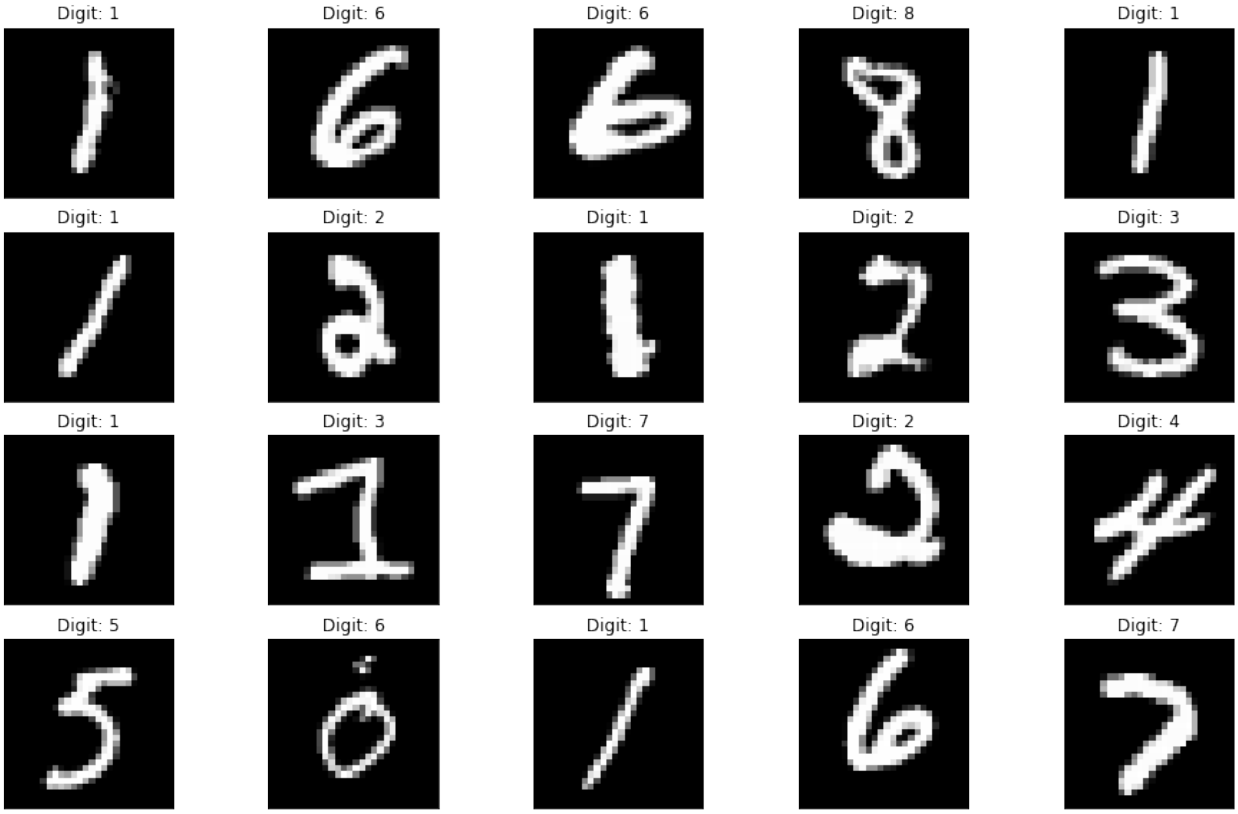
In [36]

```
plt.figure(figsize=(16, 10))
for i in range(20):
 image = random.choice(X_test).squeeze()
 digit = np.argmax(model.predict(image.reshape((1, 28, 28, 1)))[0], axis=-1)
 plot_digit(image, digit, plt, i)
plt.show()
```

Out

```
1/1 [==========] - 0s 33ms/step
1/1 [==========] - 0s 31ms/step
1/1 [==========] - 0s 31ms/step
1/1 [==========] - 0s 31ms/step
1/1 [==========] - 0s 41ms/step
1/1 [==========] - 0s 47ms/step
1/1 [==========] - 0s 31ms/step
1/1 [==========] - 0s 32ms/step
1/1 [==========] - 0s 31ms/step
1/1 [==========] - 0s 44ms/step
1/1 [==========] - 0s 31ms/step
1/1 [==========] - 0s 47ms/step
1/1 [==========] - 0s 43ms/step
1/1 [==========] - 0s 49ms/step
1/1 [==========] - 0s 33ms/step
1/1 [==========] - 0s 47ms/step
1/1 [==========] - 0s 31ms/step
1/1 [==========] - 0s 31ms/step
1/1 [==========] - 0s 31ms/step
1/1 [==========] - 0s 47ms/step
```

Out (plot)



In [37]
```
predictions = np.argmax(model.predict(X_test), axis=-1)
accuracy_score(y_test, predictions)
```

Out
```
313/313 [==========] - 2s 8ms/step
```

Out
```
0.9879
```

Estimating the model's performance

In [38]
```
score = model.evaluate(X_test, y_test, verbose=0)
print('Test loss:', score[0])    #Test Loss
print('Test accuracy:', score[1])    #Test Accuracy
```

Out
```
Test loss: 0.04982423037290573
Test accuracy: 0.9879000186920166
```

In [47]
```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = 10
epochs_range = range(epochs)
```

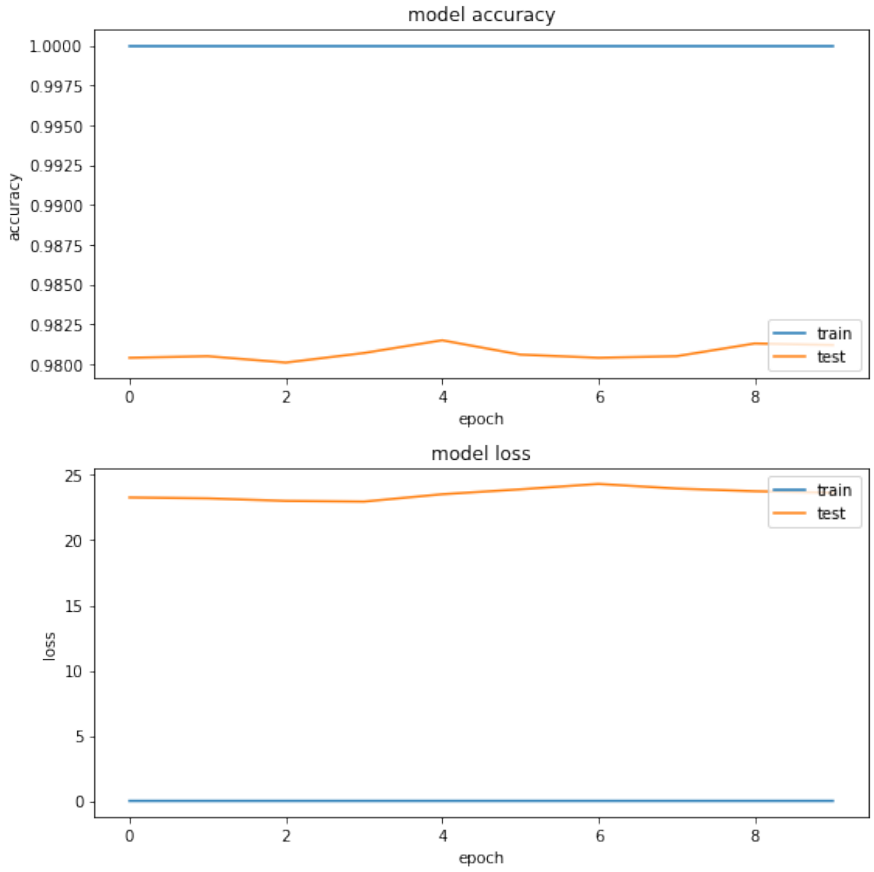
In [50]
```
import os
# plotting the metrics
fig = plt.figure()
plt.figure(figsize=(8, 8))

plt.subplot(2,1,1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='lower right')


plt.subplot(2,1,2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.title('model loss')
plt.ylabel('loss')
```

```
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper right')
plt.tight_layout()
```

Out (plot)



In [51]

```
#Save the model
# serialize model to JSON
model_digit_json = model.to_json()
with open("model_digit.json", "w") as json_file:
 json_file.write(model_digit_json)
# serialize weights to HDF5
model.save_weights("model_digit.h5")
print("Saved model to disk")
```

Out

```
Saved model to disk
```

Practical 4: Autoencoder for Anomaly Detection (Credit Card Fraud)

Notebook: DL_ASS4.ipynb **Syllabus stages:** Import required libraries; Upload / access the dataset; Encoder converts it into latent representation; Decoder converts it back to the original input; Compile the model with optimizer, loss and evaluation metrics

In [1]

```
import pandas as pd
import numpy as np
import pickle
import matplotlib.pyplot as plt
from scipy import stats
import tensorflow as tf
import seaborn as sns
from pylab import rcParams
from sklearn.model_selection import train_test_split
from keras.models import Model, load_model
from keras.layers import Input, Dense
from keras.callbacks import ModelCheckpoint, TensorBoard
from keras import regularizers

%matplotlib inline

sns.set(style='whitegrid', palette='muted', font_scale=1.5)

rcParams['figure.figsize'] = 14, 8

RANDOM_SEED = 42
LABELS = ["Normal", "Fraud"]
```

In [2]

```
df =  pd.read_csv("D:\creditcard.csv")
```

In [3]

```
count_classes = pd.value_counts(df['Class'], sort = True)
count_classes.plot(kind = 'bar', rot=0)
plt.title("Transaction class distribution")
plt.xticks(range(2), LABELS)
plt.xlabel("Class")
plt.ylabel("Frequency");
```

Out (plot)



In [4]

```
from sklearn.preprocessing import StandardScaler

data = df.drop(['Time'], axis=1)

data['Amount'] = StandardScaler().fit_transform(data['Amount'].values.reshape(-1, 1))
```

In [5]

```
X_train, X_test = train_test_split(data, test_size=0.2, random_state=RANDOM_SEED)
X_train = X_train[X_train.Class == 0]
X_train = X_train.drop(['Class'], axis=1)

y_test = X_test['Class']
X_test = X_test.drop(['Class'], axis=1)

X_train = X_train.values
X_test = X_test.values
```

In [6]

```
input_dim = X_train.shape[1]
encoding_dim = 14
```

In [7]

```
input_layer = Input(shape=(input_dim, ))

encoder = Dense(encoding_dim, activation="tanh",
                activity_regularizer=regularizers.l1(10e-5))(input_layer)
encoder = Dense(int(encoding_dim / 2), activation="relu")(encoder)

decoder = Dense(int(encoding_dim / 2), activation='tanh')(encoder)
decoder = Dense(input_dim, activation='relu')(decoder)

autoencoder = Model(inputs=input_layer, outputs=decoder)
```

In [8]

```
nb_epoch = 100
batch_size = 32
early_stop = tf.keras.callbacks. EarlyStopping( monitor= 'val_loss', min_delta=0.0001, patience=10, verbose=1, m ↵
    ode='min',
                                                restore_best_weights=True)

autoencoder.compile(optimizer='adam',
                    loss='mean_squared_error',
                    metrics=['accuracy'])

checkpointer = ModelCheckpoint(filepath="model.h5",
                               verbose=0,
                               save_best_only=True)
tensorboard = TensorBoard(log_dir='./logs',
                          histogram_freq=0,
                          write_graph=True,
                          write_images=True)

history = autoencoder.fit(X_train, X_train,
                    epochs=nb_epoch,
                    batch_size=batch_size,
                    shuffle=True,
                    validation_data=(X_test, X_test),
                    verbose=1,
                    callbacks=[checkpointer, early_stop]).history
```

Out

```
Epoch 1/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.8012 - accuracy: 0.6083 - val_loss: 0.7801 - val_accuracy: 0.6766
Epoch 2/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7373 - accuracy: 0.6875 - val_loss: 0.7631 - val_accuracy: 0.6999
Epoch 3/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7264 - accuracy: 0.7056 - val_loss: 0.7567 - val_accuracy: 0.7131
Epoch 4/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7215 - accuracy: 0.7146 - val_loss: 0.7538 - val_accuracy: 0.7193
Epoch 5/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7182 - accuracy: 0.7199 - val_loss: 0.7519 - val_accuracy: 0.7192
Epoch 6/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7161 - accuracy: 0.7231 - val_loss: 0.7509 - val_accuracy: 0.7141
Epoch 7/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7140 - accuracy: 0.7294 - val_loss: 0.7501 - val_accuracy: 0.7297
Epoch 8/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7123 - accuracy: 0.7326 - val_loss: 0.7486 - val_accuracy: 0.7180
Epoch 9/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7108 - accuracy: 0.7321 - val_loss: 0.7479 - val_accuracy: 0.7230
Epoch 10/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7094 - accuracy: 0.7336 - val_loss: 0.7472 - val_accuracy: 0.7369
Epoch 11/100
7108/7108 [==========] - 17s 2ms/step - loss: 0.7083 - accuracy: 0.7344 - val_loss: 0.7431 - val_accuracy: 0.7367
Epoch 12/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7076 - accuracy: 0.7356 - val_loss: 0.7441 - val_accuracy: 0.7267
Epoch 13/100
7108/7108 [==========] - 18s 2ms/step - loss: 0.7073 - accuracy: 0.7366 - val_loss: 0.7424 - val_accuracy: 0.7371
Epoch 14/100
7108/7108 [==========] - 17s 2ms/step - loss: 0.7066 - accuracy: 0.7381 - val_loss: 0.7424 - val_accuracy: 0.7418
Epoch 15/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7067 - accuracy: 0.7385 - val_loss: 0.7426 - val_accuracy: 0.7410
Epoch 16/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7061 - accuracy: 0.7395 - val_loss: 0.7415 - val_accuracy: 0.7403
Epoch 17/100
7108/7108 [==========] - 17s 2ms/step - loss: 0.7059 - accuracy: 0.7398 - val_loss: 0.7419 - val_accuracy: 0.7367
```

```
Epoch 18/100
7108/7108 [==========] - 17s 2ms/step - loss: 0.7063 - accuracy: 0.7403 - val_loss: 0.7416 - val_accuracy: 0.7451
Epoch 19/100
7108/7108 [==========] - 18s 3ms/step - loss: 0.7058 - accuracy: 0.7408 - val_loss: 0.7423 - val_accuracy: 0.7385
Epoch 20/100
7108/7108 [==========] - 24s 3ms/step - loss: 0.7056 - accuracy: 0.7410 - val_loss: 0.7413 - val_accuracy: 0.7402
Epoch 21/100
7108/7108 [==========] - 18s 2ms/step - loss: 0.7056 - accuracy: 0.7410 - val_loss: 0.7417 - val_accuracy: 0.7461
Epoch 22/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7049 - accuracy: 0.7420 - val_loss: 0.7407 - val_accuracy: 0.7437
Epoch 23/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7046 - accuracy: 0.7420 - val_loss: 0.7401 - val_accuracy: 0.7450
Epoch 24/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7046 - accuracy: 0.7424 - val_loss: 0.7416 - val_accuracy: 0.7364
Epoch 25/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7054 - accuracy: 0.7422 - val_loss: 0.7420 - val_accuracy: 0.7387
Epoch 26/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7045 - accuracy: 0.7420 - val_loss: 0.7418 - val_accuracy: 0.7447
Epoch 27/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7047 - accuracy: 0.7421 - val_loss: 0.7427 - val_accuracy: 0.7199
Epoch 28/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7043 - accuracy: 0.7420 - val_loss: 0.7407 - val_accuracy: 0.7459
Epoch 29/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7041 - accuracy: 0.7435 - val_loss: 0.7416 - val_accuracy: 0.7371
Epoch 30/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7046 - accuracy: 0.7428 - val_loss: 0.7407 - val_accuracy: 0.7413
Epoch 31/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7039 - accuracy: 0.7439 - val_loss: 0.7429 - val_accuracy: 0.7444
Epoch 32/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7042 - accuracy: 0.7418 - val_loss: 0.7399 - val_accuracy: 0.7418
Epoch 33/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7040 - accuracy: 0.7425 - val_loss: 0.7471 - val_accuracy: 0.7427
Epoch 34/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7039 - accuracy: 0.7437 - val_loss: 0.7403 - val_accuracy: 0.7418
Epoch 35/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7040 - accuracy: 0.7441 - val_loss: 0.7458 - val_accuracy: 0.7292
Epoch 36/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7038 - accuracy: 0.7441 - val_loss: 0.7398 - val_accuracy: 0.7354
Epoch 37/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7036 - accuracy: 0.7440 - val_loss: 0.7415 - val_accuracy: 0.7367
Epoch 38/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7036 - accuracy: 0.7440 - val_loss: 0.7399 - val_accuracy: 0.7442
Epoch 39/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7037 - accuracy: 0.7439 - val_loss: 0.7399 - val_accuracy: 0.7484
Epoch 40/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7037 - accuracy: 0.7440 - val_loss: 0.7396 - val_accuracy: 0.7472
Epoch 41/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7037 - accuracy: 0.7444 - val_loss: 0.7413 - val_accuracy: 0.7425
Epoch 42/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7037 - accuracy: 0.7439 - val_loss: 0.7399 - val_accuracy: 0.7415
Epoch 43/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7035 - accuracy: 0.7442 - val_loss: 0.7401 - val_accuracy: 0.7486
Epoch 44/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7034 - accuracy: 0.7448 - val_loss: 0.7419 - val_accuracy: 0.7386
Epoch 45/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7034 - accuracy: 0.7435 - val_loss: 0.7392 - val_accuracy: 0.7425
Epoch 46/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7038 - accuracy: 0.7442 - val_loss: 0.7396 - val_accuracy: 0.7402
Epoch 47/100
7108/7108 [==========] - 17s 2ms/step - loss: 0.7035 - accuracy: 0.7435 - val_loss: 0.7423 - val_accuracy: 0.7408
Epoch 48/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7039 - accuracy: 0.7440 - val_loss: 0.7395 - val_accuracy: 0.7465
Epoch 49/100
7108/7108 [==========] - 17s 2ms/step - loss: 0.7030 - accuracy: 0.7446 - val_loss: 0.7400 - val_accuracy: 0.7402
Epoch 50/100
7108/7108 [==========] - 18s 3ms/step - loss: 0.7034 - accuracy: 0.7435 - val_loss: 0.7404 - val_accuracy: 0.7471
Epoch 51/100
7108/7108 [==========] - 20s 3ms/step - loss: 0.7031 - accuracy: 0.7442 - val_loss: 0.7396 - val_accuracy: 0.7360
Epoch 52/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7031 - accuracy: 0.7449 - val_loss: 0.7397 - val_accuracy: 0.7436
Epoch 53/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7036 - accuracy: 0.7446 - val_loss: 0.7410 - val_accuracy: 0.7401
Epoch 54/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7031 - accuracy: 0.7450 - val_loss: 0.7379 - val_accuracy: 0.7491
Epoch 55/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7031 - accuracy: 0.7452 - val_loss: 0.7434 - val_accuracy: 0.7328
Epoch 56/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7031 - accuracy: 0.7451 - val_loss: 0.7385 - val_accuracy: 0.7459
```

Out

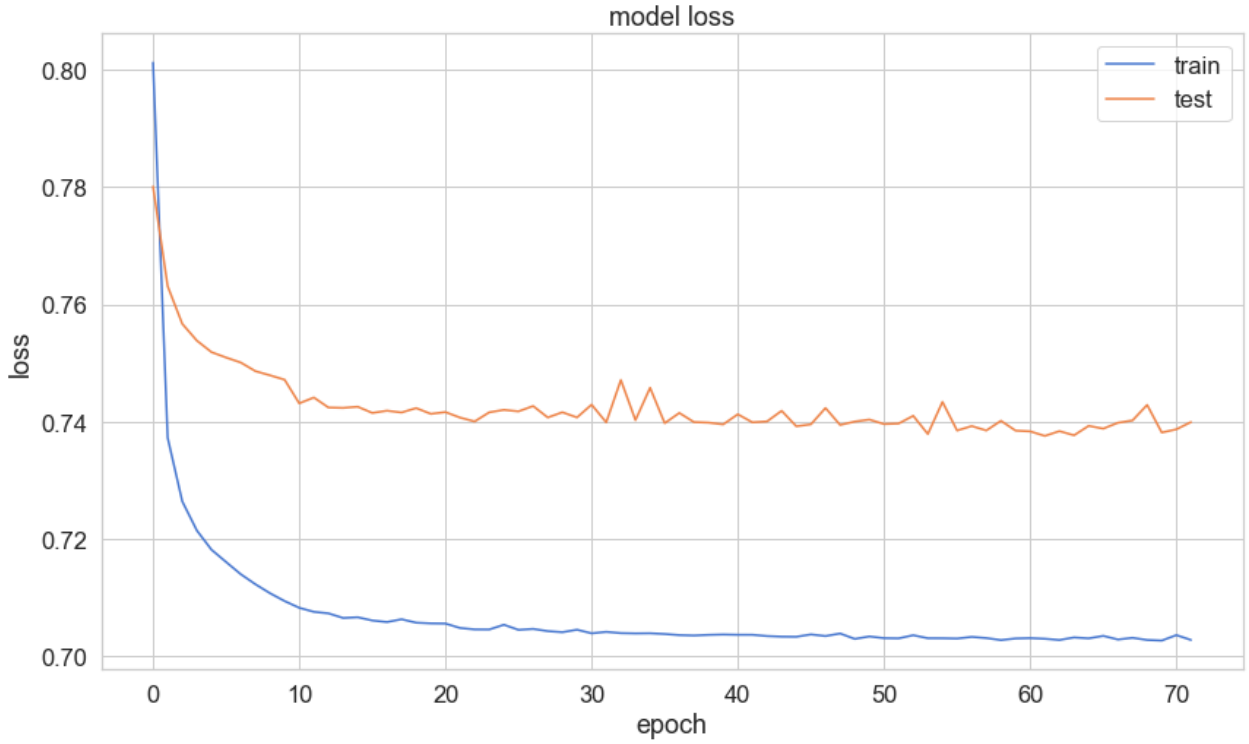
```
Epoch 57/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7033 - accuracy: 0.7443 - val_loss: 0.7393 - val_accuracy: 0.7421
Epoch 58/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7031 - accuracy: 0.7452 - val_loss: 0.7385 - val_accuracy: 0.7472
Epoch 59/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7028 - accuracy: 0.7459 - val_loss: 0.7402 - val_accuracy: 0.7470
Epoch 60/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7031 - accuracy: 0.7446 - val_loss: 0.7385 - val_accuracy: 0.7466
Epoch 61/100
7108/7108 [==========] - 17s 2ms/step - loss: 0.7031 - accuracy: 0.7457 - val_loss: 0.7384 - val_accuracy: 0.7481
Epoch 62/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7030 - accuracy: 0.7448 - val_loss: 0.7376 - val_accuracy: 0.7423
Epoch 63/100
7108/7108 [==========] - 16s 2ms/step - loss: 0.7028 - accuracy: 0.7453 - val_loss: 0.7384 - val_accuracy: 0.7442
Epoch 64/100
7108/7108 [==========] - 21s 3ms/step - loss: 0.7032 - accuracy: 0.7444 - val_loss: 0.7377 - val_accuracy: 0.7504
Epoch 65/100
```

```
7108/7108 [==========] - 20s 3ms/step - loss: 0.7031 - accuracy: 0.7455 - val_loss: 0.7393 - val_accuracy: 0.7432
Epoch 66/100
7108/7108 [==========] - 20s 3ms/step - loss: 0.7035 - accuracy: 0.7449 - val_loss: 0.7388 - val_accuracy: 0.7499
Epoch 67/100
7108/7108 [==========] - 18s 3ms/step - loss: 0.7029 - accuracy: 0.7455 - val_loss: 0.7398 - val_accuracy: 0.7365
Epoch 68/100
7108/7108 [==========] - 14s 2ms/step - loss: 0.7032 - accuracy: 0.7459 - val_loss: 0.7402 - val_accuracy: 0.7427
Epoch 69/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7028 - accuracy: 0.7460 - val_loss: 0.7429 - val_accuracy: 0.7254
Epoch 70/100
7108/7108 [==========] - 15s 2ms/step - loss: 0.7027 - accuracy: 0.7453 - val_loss: 0.7382 - val_accuracy: 0.7527
Epoch 71/100
7108/7108 [==========] - 18s 2ms/step - loss: 0.7036 - accuracy: 0.7445 - val_loss: 0.7387 - val_accuracy: 0.7468
Epoch 72/100
7108/7108 [==========] - ETA: 0s - loss: 0.7028 - accuracy: 0.7453Restoring model weights from the end of the best epoch: 62.
7108/7108 [==========] - 17s 2ms/step - loss: 0.7028 - accuracy: 0.7453 - val_loss: 0.7400 - val_accuracy: 0.7472
Epoch 72: early stopping
```

In [9]

```
plt.plot(history['loss'])
plt.plot(history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper right');
```

Out (plot)



In [10]

```
predictions = autoencoder.predict(X_test)
```

Out

```
1781/1781 [==========] - 2s 1ms/step
```

In [11]

```
mse = np.mean(np.power(X_test - predictions, 2), axis=1)
error_df = pd.DataFrame({'reconstruction_error': mse,
                         'true_class': y_test})
```

In [12]

```
error_df.describe()
```

Out

	reconstruction_error	true_class
count	56962.000000	56962.000000
mean	0.736701	0.001720
std	3.464837	0.041443
min	0.034804	0.000000
25%	0.242497	0.000000
50%	0.382573	0.000000
75%	0.602687	0.000000
max	260.987424	1.000000

In [14]

```
threshold = 50
groups = error_df.groupby('true_class')
```

```
fig, ax = plt.subplots()

for name, group in groups:
    ax.plot(group.index, group.reconstruction_error, marker='o', ms=3.5, linestyle='',
            label= "Fraud" if name == 1 else "Normal")
ax.hlines(threshold, ax.get_xlim()[0], ax.get_xlim()[1], colors="r", zorder=100, label='Threshold')
ax.legend()
plt.title("Reconstruction error for different classes")
plt.ylabel("Reconstruction error")
plt.xlabel("Data point index")
plt.show();
```

Out (plot)



In [16]

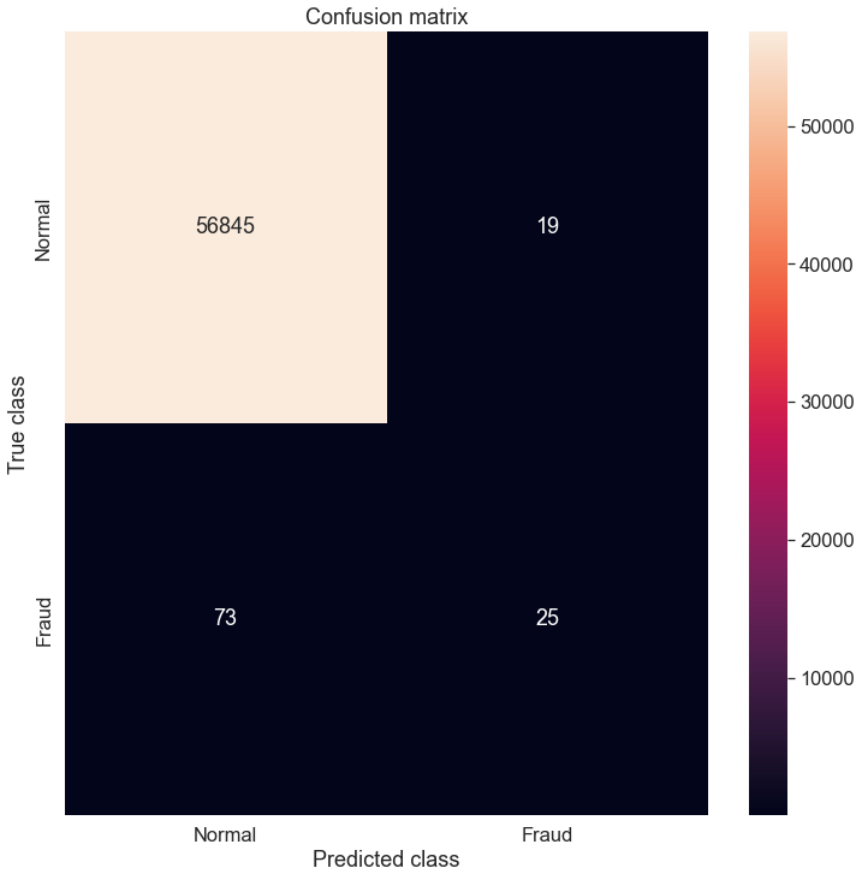
```
from sklearn.metrics import confusion_matrix,recall_score,accuracy_score,precision_score
```

In [17]

```
y_pred = [1 if e > threshold else 0 for e in error_df.reconstruction_error.values]
conf_matrix = confusion_matrix(error_df.true_class, y_pred)

plt.figure(figsize=(12, 12))
sns.heatmap(conf_matrix, xticklabels=LABELS, yticklabels=LABELS, annot=True, fmt="d");
plt.title("Confusion matrix")
plt.ylabel('True class')
plt.xlabel('Predicted class')
plt.show()
```

Out (plot)



In [21]

```
error_df['pred'] = y_pred
```

In [22]

```
# print Accuracy, precision and recall print(" Accuracy:

print("Accuracy:",accuracy_score (error_df['true_class'],error_df['pred']))
print(" Recall:",recall_score(error_df['true_class'],error_df['pred']))
print(" Precision:",precision_score(error_df['true_class'],error_df['pred']))
```

Out

```
Accuracy: 0.9983848881710614
 Recall: 0.25510204081632654
 Precision: 0.5681818181818182
```

Practical 5: Continuous Bag of Words (CBOW) Model

Notebook: DL_ASS5.ipynb **Syllabus stages:** Data preparation; Generate training data; Train model; Output

In [1]
```
import numpy as np
import keras.backend as K
from keras.models import Sequential
from keras.layers import Dense, Embedding, Lambda
from keras.utils import np_utils
from keras.preprocessing import sequence
from keras.preprocessing.text import Tokenizer
import gensim
data=open('D:\corona.txt','r')
corona_data = [text for text in data if text.count(' ') >= 2]
vectorize = Tokenizer()
vectorize.fit_on_texts(corona_data)
corona_data = vectorize.texts_to_sequences(corona_data)
total_vocab = sum(len(s) for s in corona_data)
word_count = len(vectorize.word_index) + 1
window_size = 2
print(total_vocab)
```

Out
```
198
```

In [2]
```
def cbow_model(data, window_size, total_vocab):
    total_length = window_size*2
    for text in data:
        text_len = len(text)
        for idx, word in enumerate(text):
            context_word = []
            target   = []
            begin = idx - window_size
            end = idx + window_size + 1
            context_word.append([text[i] for i in range(begin, end) if 0 <= i < text_len and i != idx])
            target.append(word)
            contextual = sequence.pad_sequences(context_word, total_length=total_length)
            final_target = np_utils.to_categorical(target, total_vocab)
            yield(contextual, final_target)
```

In [3]
```
model = Sequential()
model.add(Embedding(input_dim=total_vocab, output_dim=100, input_length=window_size*2))
model.add(Lambda(lambda x: K.mean(x, axis=1), output_shape=(100,)))
model.add(Dense(total_vocab, activation='softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam')
for i in range(10):
    cost = 0
    for x, y in cbow_model(data, window_size, total_vocab):
        cost += model.train_on_batch(contextual, final_target)
    print(i, cost)
```

Out
```
0 0
1 0
2 0
3 0
4 0
5 0
6 0
7 0
8 0
9 0
```

In [5]
```
dimensions=100
vect_file = open('D:/vectors.txt' ,'w')
vect_file.write('{} {}\n'.format(total_vocab,dimensions))
```

Out
```
8
```

In [6]
```
weights = model.get_weights()[0]
for text, i in vectorize.word_index.items():
    final_vec = ' '.join(map(str, list(weights[i, :])))
    vect_file.write('{} {}\n'.format(text, final_vec))
vect_file.close()
```

In [34]
```
cbow_output = gensim.models.KeyedVectors.load_word2vec_format('D:/vectors.txt', binary=True)
cbow_output.most_similar(positive=['the'])
```

Out

```
[('111213', 0.42092421650886536),
 ('3627341', 0.3625892996788025),
 ('.007863175', 0.36046138405799866),
 ('4006', 0.33801838755607605),
 ('.029317427', 0.2991058826446533),
 ('88', 0.2926858365535736),
 ('25402', 0.2923523187637329),
 ('02734661', 0.29116514325141907),
 ('004', 0.2871811091899872),
 ('042491533', 0.2765834629535675)]
```

In [38]

```
cbow_output['the'].shape
```

Out

```
(100,)
```

Practical 6: Transfer Learning of CNN Architectures (VGG16, tf_flowers)

Notebook: Assignemnt_6.ipynb **Syllabus stages:** Load a pre-trained CNN model trained on a large dataset; Freeze parameters (weights) in the lower convolutional layers; Add custom classifier with several layers of trainable parameters; Train classifier layers on the available training data; Fine-tune hyper parameters and unfreeze more layers as needed

In [1]
```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
```

Imports

In [2]
```
import tensorflow_datasets as tfds
import tensorflow as tf
from tensorflow.keras.utils import to_categorical
```

Out
```
c:\Users\Gaurav\AppData\Local\Programs\Python\Python39\lib\site-packages\tqdm\auto.py:22: TqdmWarning: IProgress not found. Please upd ↵
    ate jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

Load Data

In [3]
```
## Loading images and labels
(train_ds, train_labels), (test_ds, test_labels) = tfds.load("tf_flowers",
    split=["train[:70%]", "train[:30%]"], ## Train test split
    batch_size=-1,
    as_supervised=True,  # Include labels
)
```

Image Preprocessing

In [4]
```
## check existing image size
train_ds[0].shape
```

Out
```
TensorShape([442, 1024, 3])
```

In [5]
```
## Resizing images
train_ds = tf.image.resize(train_ds, (150, 150))
test_ds = tf.image.resize(test_ds, (150, 150))
```

In [6]
```
train_labels
```

Out
```
<tf.Tensor: shape=(2569,), dtype=int64, numpy=array([2, 3, 3, ..., 0, 2, 0], dtype=int64)>
```

In [7]
```
## Transforming labels to correct format
train_labels = to_categorical(train_labels, num_classes=5)
test_labels = to_categorical(test_labels, num_classes=5)
```

In [8]
```
train_labels[0]
```

Out
```
array([0., 0., 1., 0., 0.], dtype=float32)
```

Use Pretrained VGG16 Image Classification model

Load a pre-trained CNN model trained on a large dataset

In [9]
```
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
```

In [10]
```
train_ds[0].shape
```

Out
```
TensorShape([150, 150, 3])
```

In [11]
```
## Loading VGG16 model
base_model = VGG16(weights="imagenet", include_top=False, input_shape=train_ds[0].shape)
```

In [12]

```
## will not train base mode
# Freeze Parameters in model's lower convolutional layers
base_model.trainable = False
```

In [13]

```
## Preprocessing input
train_ds = preprocess_input(train_ds)
test_ds = preprocess_input(test_ds)
```

In [14]

```
## model details
base_model.summary()
```

Out

```
Model: "vgg16"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
==========
 input_1 (InputLayer)        [(None, 150, 150, 3)]     0

 block1_conv1 (Conv2D)       (None, 150, 150, 64)      1792

 block1_conv2 (Conv2D)       (None, 150, 150, 64)      36928

 block1_pool (MaxPooling2D)  (None, 75, 75, 64)        0

 block2_conv1 (Conv2D)       (None, 75, 75, 128)       73856

 block2_conv2 (Conv2D)       (None, 75, 75, 128)       147584

 block2_pool (MaxPooling2D)  (None, 37, 37, 128)       0

 block3_conv1 (Conv2D)       (None, 37, 37, 256)       295168

 block3_conv2 (Conv2D)       (None, 37, 37, 256)       590080

 block3_conv3 (Conv2D)       (None, 37, 37, 256)       590080

 block3_pool (MaxPooling2D)  (None, 18, 18, 256)       0

 block4_conv1 (Conv2D)       (None, 18, 18, 512)       1180160

 block4_conv2 (Conv2D)       (None, 18, 18, 512)       2359808

 block4_conv3 (Conv2D)       (None, 18, 18, 512)       2359808

 block4_pool (MaxPooling2D)  (None, 9, 9, 512)         0

 block5_conv1 (Conv2D)       (None, 9, 9, 512)         2359808

 block5_conv2 (Conv2D)       (None, 9, 9, 512)         2359808

 block5_conv3 (Conv2D)       (None, 9, 9, 512)         2359808

 block5_pool (MaxPooling2D)  (None, 4, 4, 512)         0

==========
Total params: 14,714,688
Trainable params: 0
Non-trainable params: 14,714,688
_________________________________________________________________
```

Add custom classifier with two dense layers of trainable parameters to model

In [15]

```
#add our layers on top of this model
from tensorflow.keras import layers, models

flatten_layer = layers.Flatten()
dense_layer_1 = layers.Dense(50, activation='relu')
dense_layer_2 = layers.Dense(20, activation='relu')
prediction_layer = layers.Dense(5, activation='softmax')


model = models.Sequential([
    base_model,
    flatten_layer,
    dense_layer_1,
    dense_layer_2,
    prediction_layer
])
```

Train classifier layers on training data available for task

In [16]

```
from tensorflow.keras.callbacks import EarlyStopping

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy'],
```

```
)
```

In [17]

```
es = EarlyStopping(monitor='val_accuracy', mode='max', patience=5,  restore_best_weights=True)
```

In [18]

```
history=model.fit(train_ds, train_labels, epochs=50, validation_split=0.2, batch_size=32, callbacks=[es])
```

Out

```
Epoch 1/50
65/65 [==========] - 91s 1s/step - loss: 1.6600 - accuracy: 0.4564 - val_loss: 1.1794 - val_accuracy: 0.5486
Epoch 2/50
65/65 [==========] - 88s 1s/step - loss: 0.8261 - accuracy: 0.6910 - val_loss: 1.0397 - val_accuracy: 0.6226
Epoch 3/50
65/65 [==========] - 90s 1s/step - loss: 0.5542 - accuracy: 0.7888 - val_loss: 0.9917 - val_accuracy: 0.6479
Epoch 4/50
65/65 [==========] - 93s 1s/step - loss: 0.3986 - accuracy: 0.8511 - val_loss: 0.9414 - val_accuracy: 0.7062
Epoch 5/50
65/65 [==========] - 95s 1s/step - loss: 0.2973 - accuracy: 0.8939 - val_loss: 0.9425 - val_accuracy: 0.7043
Epoch 6/50
65/65 [==========] - 95s 1s/step - loss: 0.2008 - accuracy: 0.9275 - val_loss: 1.0838 - val_accuracy: 0.7004
Epoch 7/50
65/65 [==========] - 96s 1s/step - loss: 0.1542 - accuracy: 0.9499 - val_loss: 1.0646 - val_accuracy: 0.7257
Epoch 8/50
65/65 [==========] - 98s 2s/step - loss: 0.1176 - accuracy: 0.9659 - val_loss: 1.0945 - val_accuracy: 0.7237
Epoch 9/50
65/65 [==========] - 99s 2s/step - loss: 0.0899 - accuracy: 0.9718 - val_loss: 1.1666 - val_accuracy: 0.7062
Epoch 10/50
65/65 [==========] - 100s 2s/step - loss: 0.0597 - accuracy: 0.9839 - val_loss: 1.2617 - val_accuracy: 0.7121
Epoch 11/50
65/65 [==========] - 100s 2s/step - loss: 0.0428 - accuracy: 0.9883 - val_loss: 1.3025 - val_accuracy: 0.6984
Epoch 12/50
65/65 [==========] - 100s 2s/step - loss: 0.0380 - accuracy: 0.9912 - val_loss: 1.3514 - val_accuracy: 0.7121
```

In [19]

```
los,accurac=model.evaluate(test_ds,test_labels)
print("Loss: ",los,"Accuracy: ", accurac)
```

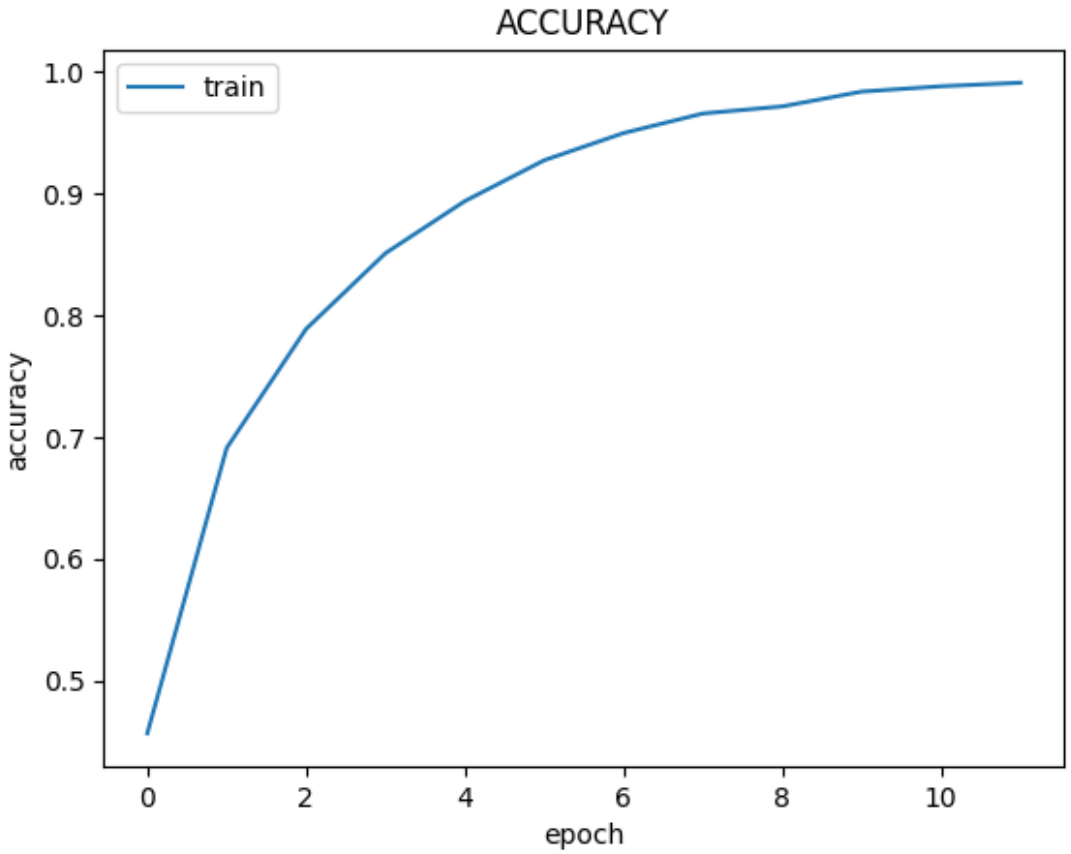
Out

```
35/35 [==========] - 44s 1s/step - loss: 0.1068 - accuracy: 0.9691
Loss:  0.10678977519273758 Accuracy:  0.9691190123558044
```

In [20]

```
import matplotlib.pyplot as plt
plt.plot(history.history['accuracy'])
plt.title('ACCURACY')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train'],loc='upper left')
plt.show()
```

Out (plot)



In [21]

```
import numpy as np
import pandas as pd
y_pred = model.predict(test_ds)
y_classes = [np.argmax(element) for element in y_pred]
#to_categorical(y_classes, num_classes=5)
#to_categorical(test_labels, num_classes=5)
print(y_classes[:10])
print("\nTest")
print(test_labels[:10])
```

Out

```
35/35 [==========] - 44s 1s/step
[2, 3, 3, 4, 3, 0, 0, 0, 0, 1]

Test
[[0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]
 [0. 0. 0. 1. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]]
```